

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
КИЇВСЬКИЙ НАЦІОНАЛЬНИЙ
УНІВЕРСИТЕТ БУДІВНИЦТВА І
АРХІТЕКТУРИ

Кафедра Кібербезпеки та комп'ютерної інженерії

ЛАБОРАТОРНА РОБОТА № 4

з курсу “ WEB-програмування ”

на тему: “Реалізація інтерактивних компонентів веб-сторінки засобами JavaScript (без використання додаткових бібліотек): маніпуляція DOM (Document Object Model), обробка подій та динамічне оновлення вмісту ”

Виконав

: група

бікс-23-1

Баранівський Андрій Валерійович

Перевірив

: Маленко М.

В.

Мета роботи:Набути практичних навичок програмування клієнтської логіки засобами мови JavaScript; освоїти API роботи з DOM: вибірку елементів, їх створення, модифікацію та видалення; навчитися реєструвати обробники подій та реагувати на дії користувача; сформувати розуміння принципів розмежування відповідальності між HTML, CSS та JavaScript.

Завдання:

Варіант	Тема (з Лаб. №1–3)	Інтерактивний компонент	події DOM	Обов'язкова JS-техніка
1	Інтернет-магазин електроніки	Фільтр товарів за категорією	click, change	querySelectorAll + classList.toggle для показу/приховування карток

Додаток 1

1. Структура HTML (Файл **index.html**)

HTML-документ створює скелет сторінки та визначає зміст магазину.

- **Мета-дані та підключення:** Використовується кодування **UTF-8**, налаштовано **viewport** для коректного відображення на мобільних пристроях. Підключено зовнішні стилі **styles.css** та скрипт **script.js** з атрибутом **defer** (завантаження після обробки HTML).
- **Семантична розмітка:**
 - **<header>**: Містить назву магазину "ElectroShop".
 - **<nav>**: Головна навігація з посиланнями на розділи сторінки.
 - **<main>**: Основний контейнер, що використовує CSS Grid для розміщення контенту та бічної панелі.
 - **<aside>**: Бокова панель з популярними категоріями та зовнішнім посиланням.
 - **<footer>**: Підвал сайту з авторськими правами.
- **Функціонал фільтрації:**
 - **#filter-container**: Містить кнопки **<button>** з атрибутами **data-category** (наприклад, **smartphones**, **laptops**), які JS використовує для розпізнавання вибору користувача.
 - **gallery**: Контейнер з картками товарів. Кожна картка має атрибут **data-cat**, який відповідає категорії товару.

2. Стилiзація CSS (Файл **styles.css**)

Оформлення робить інтерфейс сучасним та адаптивним.

- **Конфігурація (:root):** Визначено змінні для кольорів (акцентний червоний, основний темно-синій), радіусів закруглення та відступів.
- **Layout (CSS Grid):**
 - **.grid-container**: Використовує **grid-template-areas** для побудови макету ("header", "nav", "content", "sidebar", "footer").
 - **.gallery**: Використовує сітку, де кількість колонок змінюється залежно від ширини екрана (від 1 до 3).
- **Стилi фільтрів та карток:**
 - **.filter-btn**: Має заокруглені краї, анімацію при наведенні та спеціальний клас **.active** для виділення обраної категорії.
 - **.hidden**: Критично важливий клас з властивістю **display: none !important**, який

- додається через JS для приховування товарів.
 - **.card**: Має тінь, анімацію появи (**fadeIn**) та ефект підняття при наведенні курсору.
 - Адаптивність: За допомогою **@media** запитів макет перебудовується для планшетів та ПК. На великих екранах перша картка займає дві колонки (**grid-column: span 2**), якщо вона не прихована.
-

3. Логіка JavaScript (Файл **script.js**)

Скрипт забезпечує інтерактивність фільтрації товарів без перезавантаження сторінки.

- Ініціалізація: Очікує повного завантаження структури документа (**DOMContentLoaded**).
- Вибір елементів: Скрипт знаходить усі кнопки фільтрів (**.filter-btn**) та всі картки товарів (**.card**).
- Обробка подій (Event Listening):
 1. Для кожної кнопки додається слухач події **click**.
 2. При натисканні спочатку з усіх кнопок знімається клас **.active**, а на натиснуту кнопку — додається.
- Алгоритм фільтрації:
 1. Отримується значення категорії з атрибута **data-category** натиснутої кнопки.
 2. Запускається перебір усіх карток товару.
 3. Порівнюється категорія кнопки з категорією картки (**data-cat**).
 4. Умова: Якщо обрано "all" АБО категорії збігаються — у картки видаляється клас **.hidden** (вона з'являється).
 5. В іншому випадку — клас **.hidden** додається (картка зникає).

Код(index.html)

```
<!DOCTYPE html>
<html lang="uk">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>ElectroShop – Варіант 1 (Lab 4)</title>
  <link rel="stylesheet" href="styles.css">
</head>
<body>

  <header class="header-area">
    <h1>ElectroShop</h1>
  </header>

  <nav class="nav-area" aria-label="Головна навігація">
    <ul>
      <li><a href="#home">Головна</a></li>
      <li><a href="#catalog">Каталог</a></li>
      <li><a href="#specs">Характеристики</a></li>
    </ul>
  </nav>

  <main class="grid-container">
    <article class="content-area">
      <section id="home">
        <h2>Новинки смартфонів</h2>
        <p>Відкрийте для себе світ <strong>інновацій</strong> з
нашими новими моделями.</p>
        <figure>
          
          <figcaption>Флагман 2026 року</figcaption>
        </figure>
      </section>

      <section id="catalog">
        <h2>Наш асортимент</h2>

        <!-- Контейнер фільтрів (Лабораторна №4) -->
        <div id="filter-container">
          <button class="filter-btn active"
data-category="all">Всі</button>
          <button class="filter-btn"
data-category="smartphones">Смартфони</button>
          <button class="filter-btn"
data-category="laptops">Ноутбуки</button>
          <button class="filter-btn"
data-category="accessories">Акcesуари</button>
        </div>
      </section>
    </article>
  </main>
</body>
</html>
```

```
<!-- Галерея товарів з атрибутами для фільтрації -->
<div class="gallery">
  <!-- Смартфони -->
  <div class="card" style="--i: 1;" data-cat="smartphones">
    
    <h3>Смартфон Pro</h3>
    <p>55 000 грн</p>
  </div>

  <!-- Ноутбуки -->
  <div class="card" style="--i: 2;" data-cat="laptops">
    
    <h3>Ноутбук Air</h3>
    <p>42 000 грн</p>
  </div>

  <!-- Аксесуари та інше -->
  <div class="card" style="--i: 3;" data-cat="accessories">
    
    <h3>Навушники Bass</h3>
    <p>3 500 грн</p>
  </div>

  <div class="card" style="--i: 4;" data-cat="smartphones">
    
    <h3>Планшет Tab</h3>
    <p>25 000 грн</p>
  </div>

  <div class="card" style="--i: 5;" data-cat="accessories">
    
    <h3>Годинник Smart</h3>
    <p>12 000 грн</p>
  </div>

  <div class="card" style="--i: 6;" data-cat="accessories">
    
    <h3>Колонки Hi-Fi</h3>
    <p>8 900 грн</p>
  </div>

  <div class="card" style="--i: 7;" data-cat="accessories">
    
    <h3>Мишка Gaming</h3>
    <p>2 100 грн</p>
  </div>

  <div class="card" style="--i: 8;" data-cat="accessories">
```

```
        
        <h3>Клавіатура RGB</h3>
        <p>4 500 грн</p>
    </div>

    <div class="card" style="--i: 9;" data-cat="accessories">
        
        <h3>Монітор 4К</h3>
        <p>18 000 грн</p>
    </div>
</div>
</section>
</article>

<aside class="sidebar-area">
    <h3>Популярні категорії</h3>
    <p>Перегляньте наші найкращі пропозиції на аксесуари.</p>
    <a href="https://www.w3.org/" target="_blank" rel="noopener
noreferrer">Більше про нас</a>
</aside>
</main>

<footer class="footer-area">
    <p>&copy; 2026 ElectroShop. Всі права захищено.</p>
</footer>

<!-- JavaScript підключено з атрибутом defer -->
<script src="script.js" defer></script>
</body>
</html>
```

Код (styles.css)

```
/* 1. КОНФІГУРАЦІЯ */

@import
url('https://fonts.googleapis.com/css2?family=Roboto:wght@400;700&display=swap');

:root {
  --color-primary: #2C3E50;
  --color-accent: #E74C3C;
  --color-bg: #f4f7f6;
  --color-text: #333;
  --radius: 12px;
  --spacing: 1.5rem;
  --color-blue: #3498db; /* Додано для фільтрів */
}

/* 2. БАЗОВІ СТИЛІ */
* { box-sizing: border-box; }
body, h1, h2, h3, p, ul { margin: 0; padding: 0; }

body {
  font-family: 'Roboto', sans-serif;
  background-color: var(--color-bg);
  color: var(--color-text);
  line-height: 1.6;
}

img {
  max-width: 100%;
  height: auto;
  display: block;
}

h1 { font-size: clamp(1.5rem, 5vw, 2.5rem); text-align: center; }

/* 3. ГРУПКА СІТКА (CSS Grid Areas) */
.grid-container {
  display: grid;
  gap: var(--spacing);
  padding: var(--spacing);
}
```

```

max-width: 1200px;
margin: 0 auto;
grid-template-areas:
    "header"
    "nav"
    "content"
    "sidebar"
    "footer";
}

.header-area { grid-area: header; background: var(--color-primary); color:
white; padding: 2rem; border-radius: var(--radius); }
.nav-area { grid-area: nav; background: white; padding: 1rem;
border-radius: var(--radius); }
.content-area { grid-area: content; }
.sidebar-area { grid-area: sidebar; background: white; padding:
var(--spacing); border-radius: var(--radius); }
.footer-area { grid-area: footer; text-align: center; padding: 2rem;
background: #ddd; border-radius: var(--radius); }

/* Навігація */
nav ul { display: flex; justify-content: center; list-style: none; gap:
1.5rem; flex-wrap: wrap; }
nav a { text-decoration: none; color: var(--color-primary); font-weight: bold;
transition: 0.3s; }
nav a:hover { color: var(--color-accent); }

/* 4. ІНТЕРФЕЙС ФІЛЬТРАЦІЇ (ДОДАНО ДЛЯ ЛАБ №4) */

#filter-container {
    margin-bottom: var(--spacing);
    display: flex;
    justify-content: center;
    gap: 10px;
    flex-wrap: wrap;
}

.filter-btn {
    padding: 10px 20px;
    border: 2px solid var(--color-blue);
    background: transparent;

```



```
    color: var(--color-blue);
    cursor: pointer;
    border-radius: 25px;
    font-weight: bold;
    transition: 0.3s ease;
}

/* Обов'язкова JS-техніка: перемикання станів через класи */
.filter-btn:hover, .filter-btn.active {
    background: var(--color-blue);
    color: white;
}

/* Обов'язково для роботи JS-фільтра */
.hidden {
    display: none !important;
}

/* 5. ГАЛЕРЕЯ ТА КАРТКИ */
.gallery {
    display: grid;
    grid-template-columns: 1fr;
    gap: var(--spacing);
    margin-top: var(--spacing);
}

.card {
    background: white;
    padding: 1rem;
    border-radius: var(--radius);
    border: 1px solid #ddd;
    text-align: center;
    box-shadow: 0 4px 10px rgba(0,0,0,0.05);
    animation: fadeIn 0.8s ease-out both;
    transition: transform 0.3s ease;
}

.card:hover {
    transform: translateY(-5px);
    box-shadow: 0 8px 20px rgba(0,0,0,0.1);
}
```

```
}

.card img {
  width: 100%;
  height: 180px;
  object-fit: contain; /* Змінено на contain для кращого вигляду гаджетів */
  background: #fcfcfc;
  border-radius: calc(var(--radius) - 4px);
  margin-bottom: 1rem;
}

@keyframes fadeIn {
  from { opacity: 0; transform: translateY(10px); }
  to { opacity: 1; transform: translateY(0); }
}

/* 6. АДАПТИВНІСТЬ */

@media (min-width: 768px) {
  .grid-container {
    grid-template-columns: 1fr 280px;
    grid-template-areas:
      "header header"
      "nav nav"
      "content sidebar"
      "footer footer";
  }
  .gallery { grid-template-columns: repeat(2, 1fr); }
}

@media (min-width: 1024px) {
  .gallery { grid-template-columns: repeat(3, 1fr); }

  /* Умова Варіанта 1: Перша картка на 2 колонки, якщо вона не прихована фільтром */
  .card:first-child:not(.hidden) {
    grid-column: span 2;
  }
}
```

Код ([script.js](#))

```
document.addEventListener('DOMContentLoaded', () => {
    // 1. Вибираємо кнопки та КАРТКИ (у вас вони мають клас .card)
    const buttons = document.querySelectorAll('.filter-btn');
    const cards = document.querySelectorAll('.card');

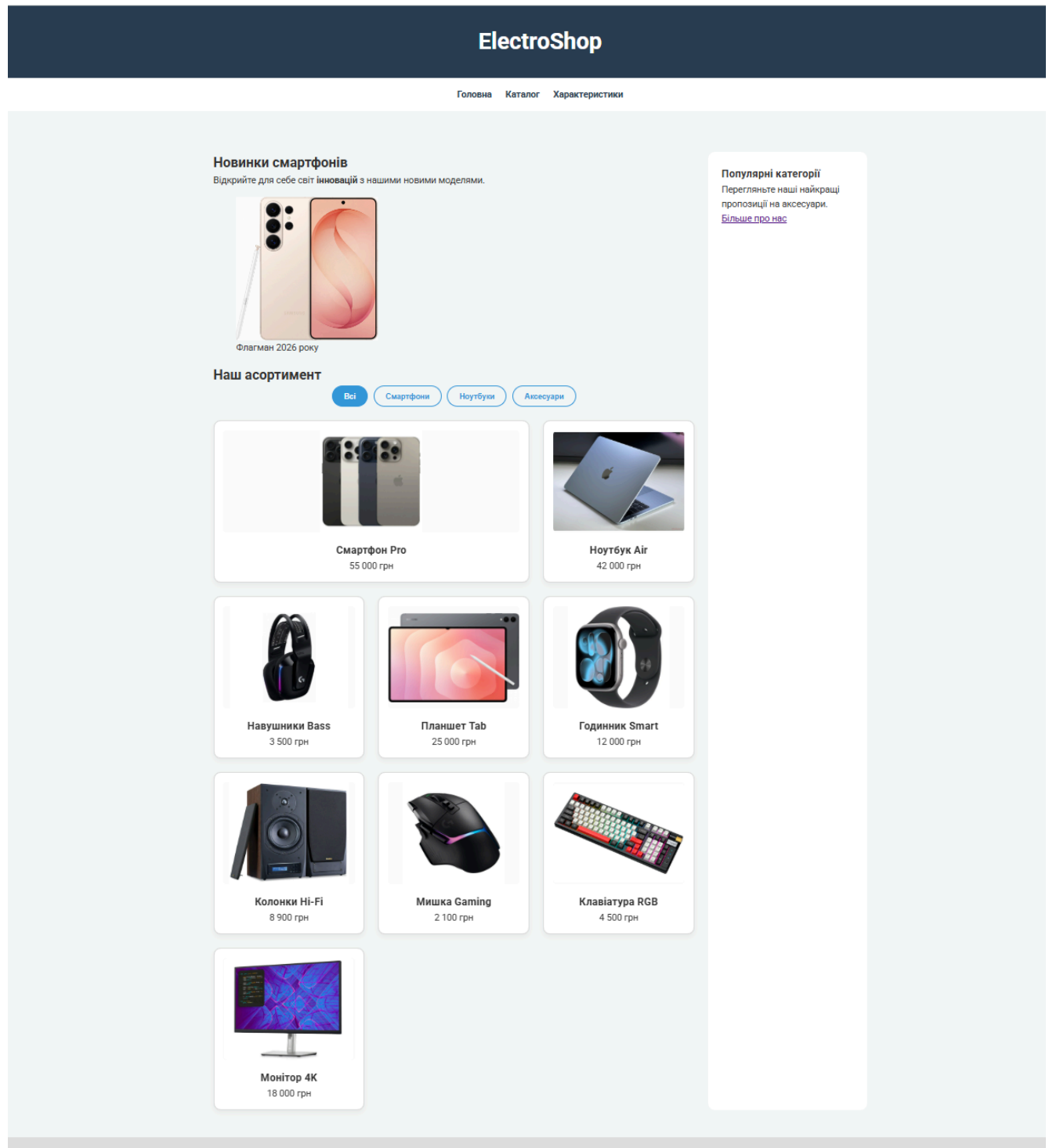
    console.log('Кількість знайдених карток:', cards.length); // Перевірка в
консолі

    buttons.forEach(button => {
        button.addEventListener('click', () => {
            // Зміна активної кнопки
            buttons.forEach(btn => btn.classList.remove('active'));
            button.classList.add('active');

            const category = button.getAttribute('data-category');
            console.log('Обрана категорія:', category);

            // 2. Логіка фільтрації
            cards.forEach(card => {
                const cardCategory = card.getAttribute('data-cat');

                // Якщо "Всі" або категорія збігається — показуємо
                if (category === 'all' || cardCategory === category) {
                    card.classList.remove('hidden');
                } else {
                    // Інакше додаємо клас .hidden
                    card.classList.add('hidden');
                }
            });
        });
    });
});
```



Відповіді на питання

1. Що таке DOM і як він пов'язаний зі структурою HTML-документа? Різниця між методами вибору елементів.

- **DOM (Document Object Model)** — це об'єктне представлення HTML-документа у вигляді дерева, де кожен тег є окремим об'єктом (вузлом), яким можна керувати через JavaScript.
- **Зв'язок:** Браузер перетворює статичний текст у `index.html` на динамічне дерево DOM, що дозволяє скриптам змінювати вміст, структуру та стилі сторінки в реальному часі.
- **document.querySelector():**
 - Повертає **перший** знайдений елемент, що відповідає вказаному CSS-селектору.
 - Якщо нічого не знайдено, повертає `null`.
- **document.querySelectorAll():**
 - Повертає **статичний список усіх** елементів (NodeList), що відповідають

селектору.

- Саме цей метод використано у `script.js` для отримання всіх кнопок `.filter-btn` та карток `.card`.

2. Різниця між `textContent` та `innerHTML`. Коли уникати `innerHTML`?

- **`textContent`:**
 - Отримує або встановлює **чистий текст** всередині вузла, ігноруючи будь-які HTML-теги.
 - Це безпечніший та швидший метод для роботи з текстом.
- **`innerHTML`:**
 - Отримує або встановлює **повний HTML-код** всередині елемента.
 - Дозволяє динамічно створювати нові теги всередині існуючих.
- **Коли уникати:** Слід уникати `innerHTML`, коли ви вставляєте дані, отримані від користувача. Це створює ризик **XSS-атак** (міжсайтового скриптингу), оскільки злоумисник може вставити шкідливий скрипт `<script>...` безпосередньо у вашу сторінку.

3. Що таке event-делегування?

- **Визначення:** Це техніка, при якій замість того, щоб вішати обробник подій на кожен окремий дочірній елемент, ми вішаємо **один обробник на їхнього спільного батька**. Подія "спливає" (bubbling) від дитини до батька, де ми її перехоплюємо.
- **Коли необхідно:** Коли на сторінці багато однотипних елементів або коли елементи додаються динамічно після завантаження сторінки.
- **Приклад:** У вашому `script.js` замість циклу `forEach` для кожної кнопки, можна було б повісити один клік на `#filter-container` і через `event.target` перевіряти, на яку саме кнопку натиснули.

4. Для чого використовується `event.preventDefault()`?

- **Призначення:** Метод скасовує **стандартну поведінку браузера** для певної події, не зупиняючи подальше розповсюдження самої події.
- **Два приклади використання:**
 1. **Форми:** Скасування перезавантаження сторінки при натисканні кнопки "Надіслати", щоб обробити дані через JavaScript.
 2. **Посилання:** Скасування переходу за адресою `href` при кліку на посилання (наприклад, щоб відкрити модальне вікно замість переходу на нову сторінку).

5. Різниця між методами `classList`.

Ці методи використовуються для маніпуляції CSS-класами, як це зроблено у вашому проєкті зі стилями `active` та `hidden`:

- **`classList.add()`:** Додає один або кілька класів. Доцільно, коли потрібно гарантовано приховати елемент (наприклад, додавання `.hidden`).
- **`classList.remove()`:** Видаляє класи. Використовується для активації елементів (наприклад, видалення `.hidden` у `script.js`).
- **`classList.toggle()`:** Додає клас, якщо його немає, і видаляє, якщо він є. Ідеально підходить для перемикачів (наприклад, кнопка "Темна тема" або "Показати/Сховати деталі").

6. Що таке `localStorage`, його обмеження та типи даних?

- **localStorage**: Це об'єкт веб-сховища, який дозволяє зберігати дані в браузері користувача **без терміну дії**. Дані зберігаються навіть після закриття вкладки або перезапуску браузера.
- **Обмеження**:
 - **Об'єм**: Зазвичай обмежений ~5-10 МБ на домен.
 - **Синхронність**: Операції читання/запису блокують основний потік виконання (не підходить для величезних об'ємів даних).
 - **Безпека**: Доступний для будь-якого скрипта на сторінці (не можна зберігати паролі або токени).
- **Типи даних**: Підтримує лише **рядки (strings)**. Щоб зберегти об'єкт або масив, їх потрібно перетворити на рядок через **JSON.stringify()**, а при читанні — назад через **JSON.parse()**.